

React Day 2 — Complete Interview Notes

Today you worked on the **Staff Management System** and connected your React frontend with your Spring Boot + MySQL backend. These notes cover the concepts you actually practiced today.

1. Controlled Components

A **controlled component** is a form input whose value is controlled by React state.

Example:

```
const [formData, setFormData] = useState({
  name: "",
  department: "",
  role: ""
});
```

Input:

```
<input
  name="name"
  value={formData.name}
  onChange={handleChange}
/>
```

Here:

```
Input
↓
onChange
↓
handleChange()
↓
setFormData()
↓
React state changes
↓
Input displays updated value
```

Interview answer

A controlled component is a form element whose value is controlled by React state. The input value is stored in state and updated using an event handler such as onChange.

2. formData

You used:

```
const [formData, setFormData] = useState({
  name: "",
  department: "",
  role: ""
});
```

formData contains the current values entered into the form.

For example:

```
{
  name: "Aniket",
  department: "Cardiology",
  role: "Doctor"
}
```

setFormData updates that data.

3. handleChange()

You used:

```
function handleChange(e) {
  const { name, value } = e.target;

  setFormData({
    ...formData,
    [name]: value
  });
}
```

Important concepts

`e.target` → the input element that triggered the event.

```
e.target.name
```

gets:

```
name
```

and:

```
e.target.value
```

gets the value entered by the user.

Why `[name]`?

Because you have multiple inputs:

```
name="name"  
name="department"  
name="role"
```

So:

```
[name]: value
```

dynamically updates the correct property.

4. Spread Operator

You used:

```
setFormData({  
  ...formData,  
  [name]: value  
});
```

The spread operator:

```
...formData
```

copies the existing properties.

For example:

```
formData = {  
  name: "Aniket",  
  department: "Emergency",  
  role: "Doctor"  
}
```

If the user changes department:

```
[name]: value
```

becomes:

```
department: "Cardiology"
```

Result:

```
{  
  name: "Aniket",  
  department: "Cardiology",  
  role: "Doctor"  
}
```

Interview answer

I use the spread operator to preserve the existing form state while updating only the field that changed.

5. Form Submission

You used:

```
<form onSubmit={handleSubmit}>
```

and:

```
function handleSubmit(e) {  
  e.preventDefault();  
}
```

Why preventDefault()?

Normally, submitting an HTML form can reload/navigate the page.

React handles the submission itself, so:

```
e.preventDefault();
```

prevents the browser's default form submission behavior.

Interview answer

preventDefault() prevents the browser's default form submission and page reload so that React can handle the submission using JavaScript.

6. Form Validation

You created:

```
const newErrors = {};
```

Then:

```
if (!formData.name.trim()) {  
  newErrors.name = "Name is required";  
}
```

Similarly:

```
if (!formData.department.trim()) {  
  newErrors.department = "Department is required";  
}  
  
if (!formData.role.trim()) {  
  newErrors.role = "Role is required";  
}
```

Then:

```
if (Object.keys(newErrors).length > 0) {  
  setErrors(newErrors);  
  return;  
}
```

This prevents the API request when required fields are empty.

7. POST Request

For adding staff, you used:

```
fetch("http://localhost:8080/api/staff", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify(formData)  
});
```

Important parts

POST:

Used to send data to the backend to create a new staff member.

Content-Type:

application/json

tells the backend that the request body contains JSON.

JSON.stringify():

JSON.stringify(formData)

converts the JavaScript object into JSON text that can be sent in the HTTP request.

8. async/await

You used:

```
async function handleSubmit(e) {
```

and:

```
const response = await fetch(...);
```

await waits for the asynchronous operation to complete.

Without await, the result of fetch() is not immediately available.

Interview answer

I use `async/await` to handle asynchronous API requests in a cleaner and more readable way.

9. Handling API Response

You used:

```
if (!response.ok) {  
  throw new Error("Request failed");  
}
```

This checks whether the HTTP response was successful.

Then:

```
const data = await response.json();
```

converts the JSON response into a JavaScript object.

10. Props

Your `Staff` component receives:

```
<Staff  
  id={staff.id}  
  name={staff.name}  
  department={staff.department}  
  role={staff.role}  
  onEdit={handleEdit}  
  onDelete={handleDelete}  
>
```

These are **props**.

Inside `Staff`:

```
function Staff(props) {
```

You access them using:

```
props.name  
props.department  
props.role  
props.id
```

Interview answer

Props are used to pass data and functions from a parent component to a child component.

11. Callback Function — Child to Parent Communication

This was an important concept you learned today.

Parent:

```
<Staff onDelete={handleDelete} />
```

Child:

```
<button onClick={() => props.onDelete(props.id)}>  
  Delete  
</button>
```

The child doesn't directly modify the parent's state.

Instead:

```
Child  
↓  
calls callback  
↓  
Parent function  
↓  
Parent changes state
```

Interview answer

React follows one-way data flow. A parent can pass a callback function to a child through props, allowing the child to communicate an event back to the parent.

12. Edit Functionality

You created:

```
const [editingStaff, setEditingStaff] = useState(null);
```

Initially:

```
editingStaff = null
```

When the user clicks Edit:

```
function handleEdit(id) {  
  const staff = staffList.find((staff) => staff.id === id);  
  setEditingStaff(staff);  
}
```

13. find()

You used:

```
staffList.find((staff) => staff.id === id);
```

find() searches the array and returns the first matching object.

Example:

```
[  
  { id: 3, name: "Aniket" },  
  { id: 4, name: "Jay" }  
]
```

If:

```
id = 3
```

then:

```
find()
```

returns:

```
{ id: 3, name: "Aniket" }
```

14. `useEffect()` for Edit

You used:

```
useEffect(() => {  
  if (editingStaff) {  
    setFormData({  
      name: editingStaff.name,  
      department: editingStaff.department,  
      role: editingStaff.role  
    });  
  }  
}, [editingStaff]);
```

This means:

Whenever `editingStaff` changes, populate the form with that staff member's data.

Flow:

```
Click Edit  
  ↓  
handleEdit(id)  
  ↓  
find staff  
  ↓  
setEditingStaff(staff)  
  ↓  
editingStaff changes  
  ↓  
useEffect()  
  ↓  
setFormData()  
  ↓  
Form populated
```

15. Reusing the Form for Add and Edit

This is one of the best concepts from today's project.

You used:

```
{editingStaff ? "Update Staff" : "Add Staff"}
```

So the same form handles both operations.

Add mode

```
editingStaff = null
```

Button:

Add Staff

Edit mode

```
editingStaff = staff object
```

Button:

Update Staff

16. POST vs PUT

Your form determines which operation is required.

```
let url = "http://localhost:8080/api/staff";  
let method = "POST";
```

```
if (editingStaff) {  
  url = `http://localhost:8080/api/staff/${editingStaff.id}`;  
  method = "PUT";  
}
```

Add

POST /api/staff

Update

PUT /api/staff/3

This allows one form to perform both operations.

17. PUT Request

For editing:

PUT /api/staff/3

with JSON:

```
{  
  "name": "Aniket",  
  "department": "Cardiology",  
  "role": "Nurse"  
}
```

Backend updates the existing record.

Interview answer

PUT is used in our application to update an existing staff member identified by their ID.

18. Updating React State After PUT

After the backend returns the updated staff:

```
function handleStaffUpdated(updatedStaff) {  
  setStaffList(  
    staffList.map((staff) =>  
      staff.id === updatedStaff.id  
        ? updatedStaff  
        : staff  
    )  
  );  
}
```

This uses map().

How it works

If IDs match:

```
staff.id === updatedStaff.id
```

replace the old object.

Otherwise:

```
: staff
```

keep the existing object.

Example:

Old:

Aniket → Emergency → Doctor

PUT

New:

Aniket → Cardiology → Nurse

React state becomes:

Aniket → Cardiology → Nurse

without refreshing the page.

19. `map()` for Updating

Remember:

```
staffList.map(...)
```

creates a **new array**.

This is important because React state should be updated without directly modifying the existing array.

20. `filter()` for Delete

Your delete implementation used:

```
setStaffList(  
  staffList.filter((staff) => staff.id !== id)  
);
```

Suppose:

1 Rahul
2 Aniket
3 Jay

Delete ID 2.

filter() keeps:

1 Rahul
3 Jay

and removes:

2 Aniket

Interview answer

I use filter() to create a new staff array without the deleted staff member and then update the React state.

21. DELETE Request

You implemented:

```
const response = await fetch(  
  `http://localhost:8080/api/staff/${id}`,  
  {  
    method: "DELETE"  
  }  
);
```

Flow:

Click Delete
↓
handleDelete(id)
↓
DELETE /api/staff/{id}
↓
Spring Boot
↓
Repository

↓
MySQL

Then:

```
setStaffList(  
  staffList.filter((staff) => staff.id !== id)  
);
```

updates the frontend.

22. GET Request

You already implemented GET during Day 1/Day 2 integration:

```
useEffect(() => {  
  fetch("http://localhost:8080/api/staff")  
    .then(response => response.json())  
    .then(data => {  
      setStaffList(data);  
    });  
}, []);
```

This loads staff from Spring Boot when the component mounts.

23. Loading State

You used:

```
const [loading, setLoading] = useState(true);
```

Initially:

```
loading = true
```

After successful API request:

```
setLoading(false);
```

UI:

```
{loading && <p>Loading staff...</p>}
```

So the user sees:

Loading staff...

while the request is running.

24. Error State

You used:

```
const [error, setError] = useState("");
```

If the API fails:

```
setError("Failed to load staff");
```

Then:

```
{error && <p>{error}</p>}
```

displays the error.

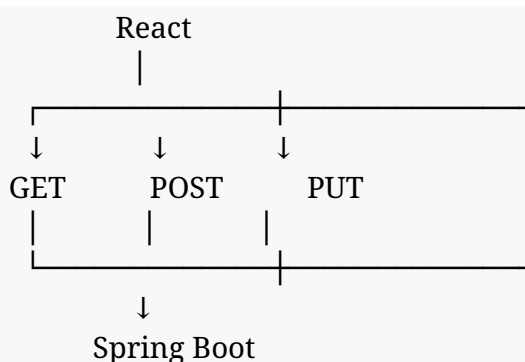
Interview answer

Loading and error states improve the user experience by showing the current status of an asynchronous API request.

25. Complete CRUD Flow

This is **very important for your interview**.

Your Staff Management System now has:



↓
JPA Repository
↓
Hibernate
↓
MySQL

And:

DELETE
↓
Spring Boot
↓
JPA Repository
↓
Hibernate
↓
MySQL

26. Full Project Flow — Interview Answer

If interviewer asks:

“Explain your React application.”

You can say:

I built a Staff Management System using React for the frontend and Spring Boot with MySQL for the backend. React maintains the staff list and form data using `useState`. I use `useEffect` to fetch staff data from the Spring Boot REST API when the component loads. The form is a controlled component, so its values are managed through React state. I implemented CRUD operations using GET, POST, PUT and DELETE APIs. For editing, I store the selected staff member in `editingStaff` state and populate the form using `useEffect`. After API operations, I update the React state using methods such as `map()` and `filter()` so the UI updates without a page refresh.

That’s a **strong project explanation** for your current level.

27. Important Interview Questions

Q1. What is a controlled component?

A form input whose value is controlled by React state.

Q2. Why use `e.preventDefault()`?

To prevent the browser's default form submission and page reload.

Q3. What is `e.target.value`?

It gets the current value of the input that triggered the event.

Q4. Why use the spread operator in `setFormData()`?

To preserve the existing form fields while updating the field that changed.

Q5. What is the purpose of `editingStaff`?

It stores the staff member currently being edited and determines whether the form is in Add or Update mode.

Q6. Why use `find()`?

To locate a specific staff object from the staff array using its ID.

Q7. Why use `map()` during update?

To create a new array where the updated staff replaces the old staff object.

Q8. Why use `filter()` during delete?

To create a new array excluding the staff member that was deleted.

Q9. How does a child component communicate with its parent?

The parent passes a callback function through props, and the child calls that callback.

Q10. Why use `async/await`?

To handle asynchronous API requests in a readable way.

Q11. What is the difference between POST and PUT?

POST is used to create a new resource, while PUT is used to update an existing resource.

Q12. How does React communicate with Spring Boot?

React sends HTTP requests using `fetch()` to Spring Boot REST endpoints and receives JSON responses.

28. Your Most Important Code Patterns

Memorize these patterns rather than memorizing the entire project.

Controlled input

```
<input
  name="name"
  value={formData.name}
  onChange={handleChange}
/>
```

Change handler

```
function handleChange(e) {
  const { name, value } = e.target;

  setFormData({
    ...formData,
    [name]: value
  });
}
```

POST

```
fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(formData)
});
```

PUT

```
fetch(`/api/staff/${id}`, {
  method: "PUT",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(formData)
});
```

```
},  
  body: JSON.stringify(formData)  
});
```

DELETE

```
fetch(`/api/staff/${id}`, {  
  method: "DELETE"  
});
```

Update state

```
setStaffList(  
  staffList.map(staff =>  
    staff.id === updatedStaff.id  
      ? updatedStaff  
      : staff  
  )  
);
```

Delete from state

```
setStaffList(  
  staffList.filter(staff => staff.id !== id)  
);
```

Day 2 Interview Revision — What You Must Be Able to Explain

Before moving forward, make sure you can explain these **without looking at your notes**:

1. Controlled Component
2. formData
3. handleChange
4. e.target
5. e.target.value
6. Spread operator
7. preventDefault()
8. Form validation
9. Props

10. Callback functions
11. Child → Parent communication
12. async/await
13. fetch()
14. POST
15. PUT
16. DELETE
17. editingStaff
18. useEffect for editing
19. find()
20. map()
21. filter()
22. Loading state
23. Error state
24. React → Spring Boot API flow
25. Complete CRUD flow

Most important for your placement interviews

Don't just memorize definitions. Be able to **open your Staff Management project and explain why you wrote each piece of code.**

Your project is now a good practical example of:

React + REST API + Spring Boot + JPA/Hibernate + MySQL CRUD.